

Notes Week 9

a. Validating Tides Implementation

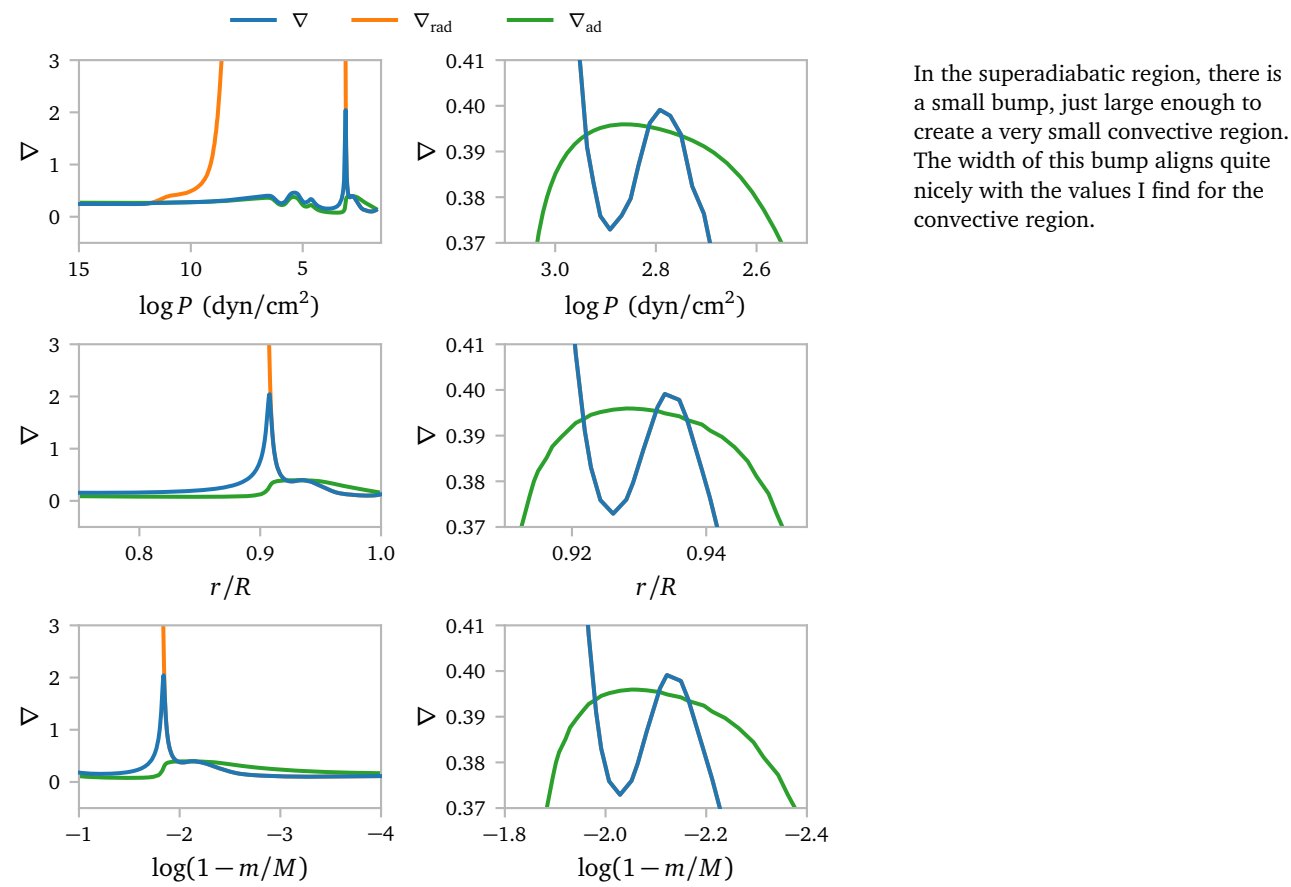
In the output, the convective turnover timescale was not changing. I traced this bug down to the following line of code:

```
t_conv = 0.4311 * (3 * M_conv * R_conv**2 / (s% photosphere_L))**(1/3) * secyer
```

Here, the integer division in the exponent leads to $x^0 = 1$ always.
Another problem revealed itself when analysing the output.

```
R_top = 380.72570878310302
R_bot = 365.63276071638143
R_conv= 15.092948066721579
in compute_t_conv: M_conv = 1.7530753171383301E-003
in compute_t_conv: R_conv = 15.092948066721579
in compute_t_conv: L = 8637.1120791303983
5.1764712937202029E-002
M_conv= 1.7530753171383301E-003 Msun
R_conv= 15.092948066721579 Rsun
t_conv= 704232.07225991588 s
t_conv= 0.26778921296673353 months
k_over_T = 7.7600225066267609E-013 /s
Omega_star = 2.6920249879859447E-008 /s
Omega_orb = 2.4471986527612926E-008 /s
J_orb = 3.5784679676889668E+053 g cm^2 / s
a = 91940611174289.297 cm
dt = 888553195.76110470 s
I_eff = 2.8898415232377608E+059 g cm^2
rg2 = 0.10857893011391405
```

The convective regions give wrong results, likely due to the superadiabicity of the outer layers looking at the minimal mass involved in the convective region.
Below, I plot the entropy profile of a star where my code finds a very tiny convective mass and radius:



Therefore, I should adjust the methods to find the mass and radius of the convective envelope region. My idea is to look for *all* convective regions that start sufficiently close to the outer edge of the star, and figure out which is the largest. I use this one for the final result.

```
real(dp) function compute_M_conv(s) result(M_conv)
implicit none

type(star_info), pointer :: s
integer :: i
real(dp) :: q_top, q_bot, q_max

M_conv = 0
q_max = 0
do i = 1, s% n_conv_regions
if (abs(s% cz_top_mass(i) / Msun / s% star_mass - 1d0) < 3d-2) then
! --- starts sufficiently close to the edge
q_top = s% cz_top_mass(i) / Msun / s% star_mass
q_bot = s% cz_bot_mass(i) / Msun / s% star_mass

write(*,*) q_top, q_bot

if (q_top - q_bot > q_max) then
q_max = q_top - q_bot
write(*,*) "found new q_max:"
write(*,*) q_top, q_bot, q_max
end if

end if
end do
write(*,*) "found M_conv:"
write(*,*) q_max, "gives M_conv = ", s% mstar * q_max / Msun

M_conv = s% mstar * q_max / Msun

end function compute_M_conv

real(dp) function compute_R_conv(s) result(R_conv)
implicit none

type(star_info), pointer :: s
real(dp) :: R_bot, R_top
real(dp) :: q_top, q_bot, q_max
real(dp) :: q_top_max, q_bot_max
integer :: k, i

R_conv = 0

! --- first find q_top and q_bot

q_max = 0
q_top_max = 0
q_bot_max = 0

do i = 1, s% n_conv_regions
if (abs(s% cz_top_mass(i) / Msun / s% star_mass - 1d0) < 3d-2) then
! --- starts sufficiently close to the edge
q_top = s% cz_top_mass(i) / Msun / s% star_mass
q_bot = s% cz_bot_mass(i) / Msun / s% star_mass

if (q_top - q_bot > q_max) then
q_max = q_top - q_bot
q_top_max = q_top
q_bot_max = q_bot
end if
end if
end do

! --- now find which rs this corresponds to
R_top = 0d0

do k = 1, s% nz
if (s% m(k) / Msun / s% star_mass <= q_top_max) then
R_top = s% r(k)
exit
end if
end do

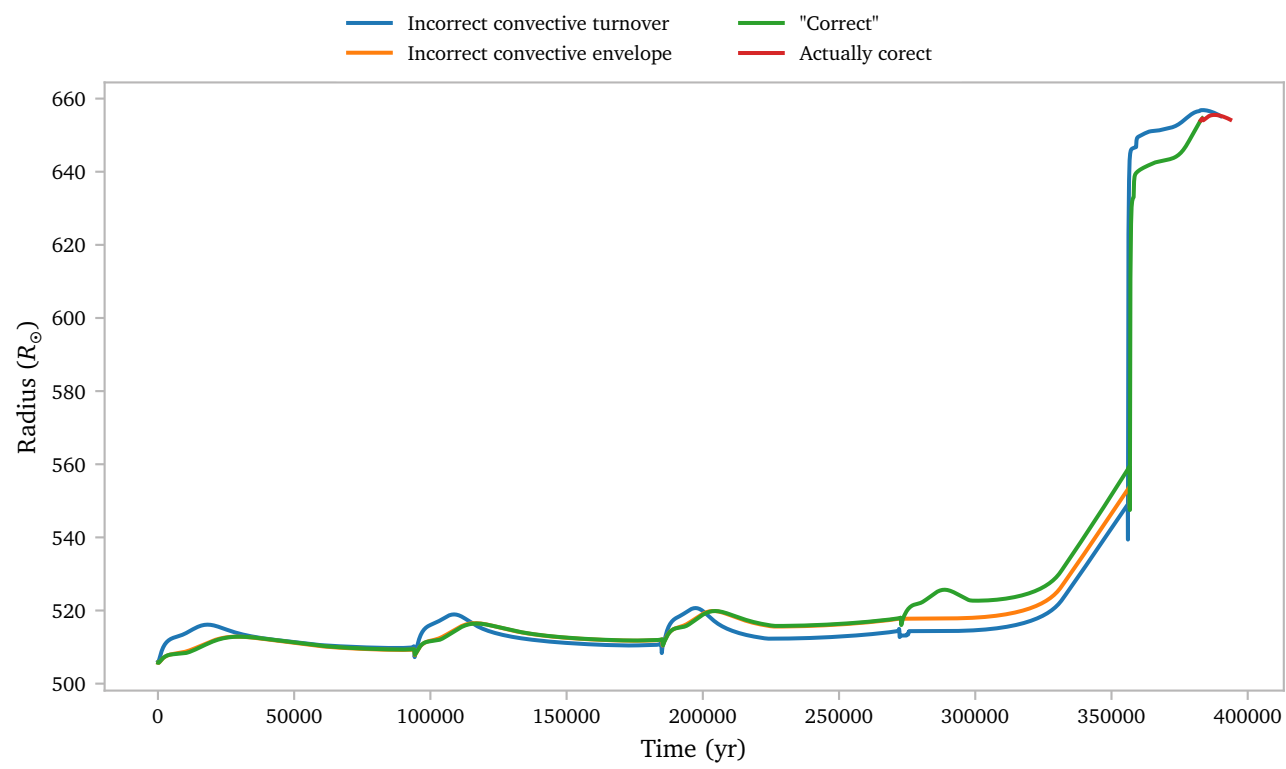
R_bot = 0d0 ! for safety

do k = k, s% nz
if (s% m(k) / Msun / s% star_mass <= q_bot_max) then
R_bot = s% r(k)
exit
end if
end do

R_conv = (R_top - R_bot) / Rsun

end function compute_R_conv
```

This results in the "correct" convective regions¹.1 At least, that is what I thought.



The "Correct" tide model terminates before the envelope is fully stripped. Again, my tides implementation cannot find the correct size of the convective envelope.

```

rl_relative_gap for each component
↳ NaN NaN NaN
rl for each component NaN
↳ NaN
r for each component 35119709581892.098
↳ 0.000000000000000000
m for each component 1.4776735316624046E+033
↳ 4.2199201770478900E+033
separation, angular momentum NaN
↳ NaN
jdot, jdot_mb, jdot_gr, jdot_ml:
↳ NaN 0.000000000000000000
↳ -1.2689479219736663E+024 -9.0373276853257355E+040
jdot_ls, jdot_missing_wind, extra_jdot:
↳ NaN 0.000000000000000000 0.000000000000000000
error solving rl_rel_gap
retry 21509
found M_conv:
0.000000000000000000 gives M_conv =
↳ 0.000000000000000000
M_conv= 0.000000000000000000 Msun
R_conv= 0.000000000000000000 Rsun
t_conv= 0.000000000000000000 s
t_conv= 0.000000000000000000 months
k_over_T = NaN /s
Omega_star = NaN /s
Omega_orb = 1.0050389612588993E-008 /s
J_orb = 2.6621375288468520E+053 g cm^2 / s
a = 155565622649979.09 cm
dt = 91137198.467318997 s
I_eff = 8.4459593589120531E+058 g cm^2
rg2 = 4.6331363281327982E-002

```

In my code, I do

```

do i = 1, s% n_conv_regions
  if (abs(s% cz_top_mass(i) / Msun / s% star_mass - 1d0) < 3d-2) then
    ! --- find envelope

```

to find envelope regions. This however, fails when the envelope becomes small, as relative to the star mass, the superadiabatic region grows and the convective envelope starts deeper into the star, surpassing $0.03M_{\text{star}}$. Really, we should just check the largest convective region that is NOT the core. For TPAGB-stars, the core is radiative, so we could probably just take the largest convective region. However, for the sake of generality, I still want to implement the core criterion. We can define the core mass using the mesa variable `s% he_core_mass`. Then, the new check is trivial:

```

do i = 1, s% n_conv_regions
  if (s% cz_top_mass(i) / Msun - s% he_core_mass > 8d-3) then
    ! --- find envelope

```

Here, I use `0.008` as a minimum mass above which the envelope should start. I think this is fine, since the simulation only goes down to $0.02M_{\odot}$.

This results in the red line from the plot above. The discontinuity arises from the sudden jump in convective envelope mass, but the important thing is that the model terminates due to a stripped envelope.

These are the typical values of the tides at the start of the evolution:

```

M_conv= 1.4104929944399043 Msun
R_conv= 346.63684044545323 Rsun
t_conv= 51634029.747670218 s
t_conv= 19.634204026036283 months
k_over_T = 2.1318031522939941E-009 /s
Omega_star = 2.4405051105719117E-008 /s
Omega_orb = 2.6326289142027669E-008 /s
J_orb = 3.6267648430630879E+053 g cm^2 / s
a = 88316477043967.516 cm
dt = 2524608000.0000000 s
I_eff = 2.7455090070529471E+059 g cm^2
rg2 = 0.11698324528595912

```

And these are typical values at the end of the evolution:

```

M_conv= 1.9333378562878496E-002 Msun
R_conv= 275.58270883774799 Rsun
t_conv= 11041685.311094698 s
t_conv= 4.1986787250341084 months
k_over_T = 2.8328298198011495E-010 /s
Omega_star = 1.7601613982742538E-008 /s
Omega_orb = 9.2259421941393170E-009 /s
J_orb = 2.3516310877365511E+053 g cm^2 / s
a = 162522837053206.97 cm
dt = 31346875.096955534 s
I_eff = 9.0310976468297850E+057 g cm^2
rg2 = 1.5792074851643058E-002

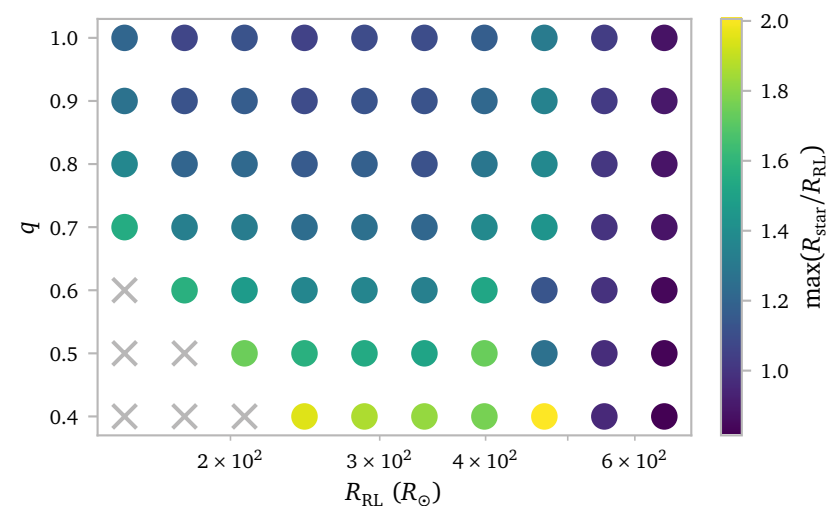
```

I think (?) these values look alright. The change in convective envelope mass makes sense due to the mass loss from RLOF and wind. With similar radii, this decreases the convective turnover timescale as well. Due to RLOF, the orbit expands drastically which decreases the orbital frequency and increases a . To me it also makes sense that the moment of inertia decreases, as is lost from the system while maintaining similar radii. This also describes why the gyration radius decreases.

b. Investigating Curiosities TPAGB-grid

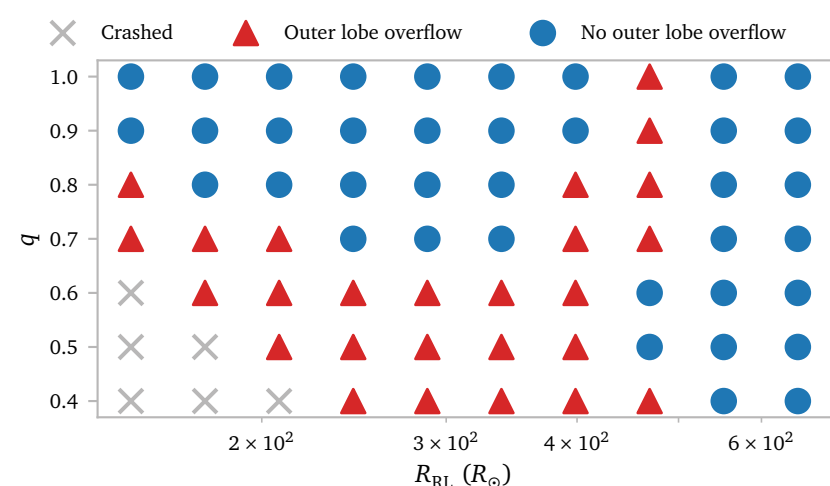
Last week, we saw some models that did not follow the general trends of the rest of the models. Specifically, there were some models that took on smaller ratios of $R_{\text{star}}/R_{\text{RL}}$ than expected.

To reproduce this in another way, we can plot all models, and show the maximum fraction of the Roche lobe radius.



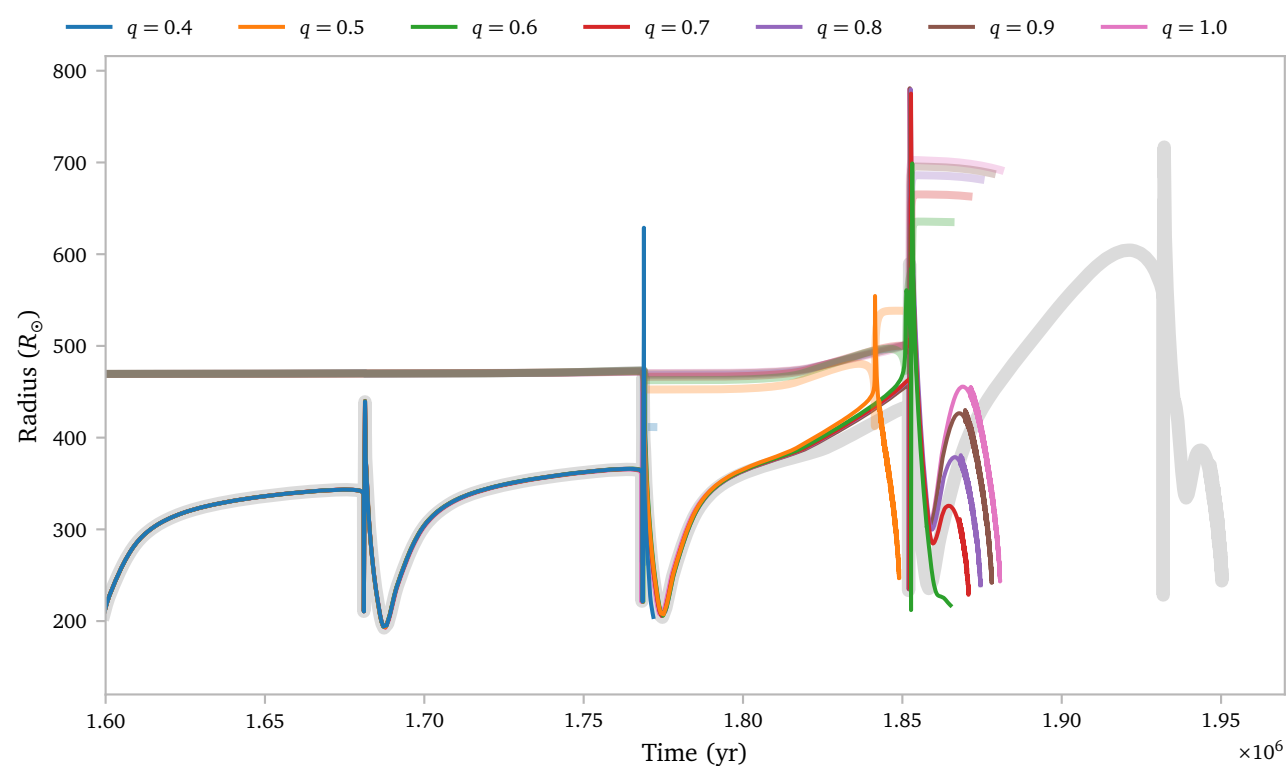
There is some structure visible here, but I find it difficult to clearly distinguish the structure. It might be better to categorize the systems instead.

Below, I show a categorized version of the plot above.



Now the structure is apparent. Most interestingly, there seem to be two systems with $R_{\text{RL}} \approx 500$ that have a lower ratio $R_{\text{star}}/R_{\text{RL}}$. These are exactly the two systems we saw last week. Then, the last two columns all have low $R_{\text{star}}/R_{\text{RL}}$ ratios. We should investigate this too.

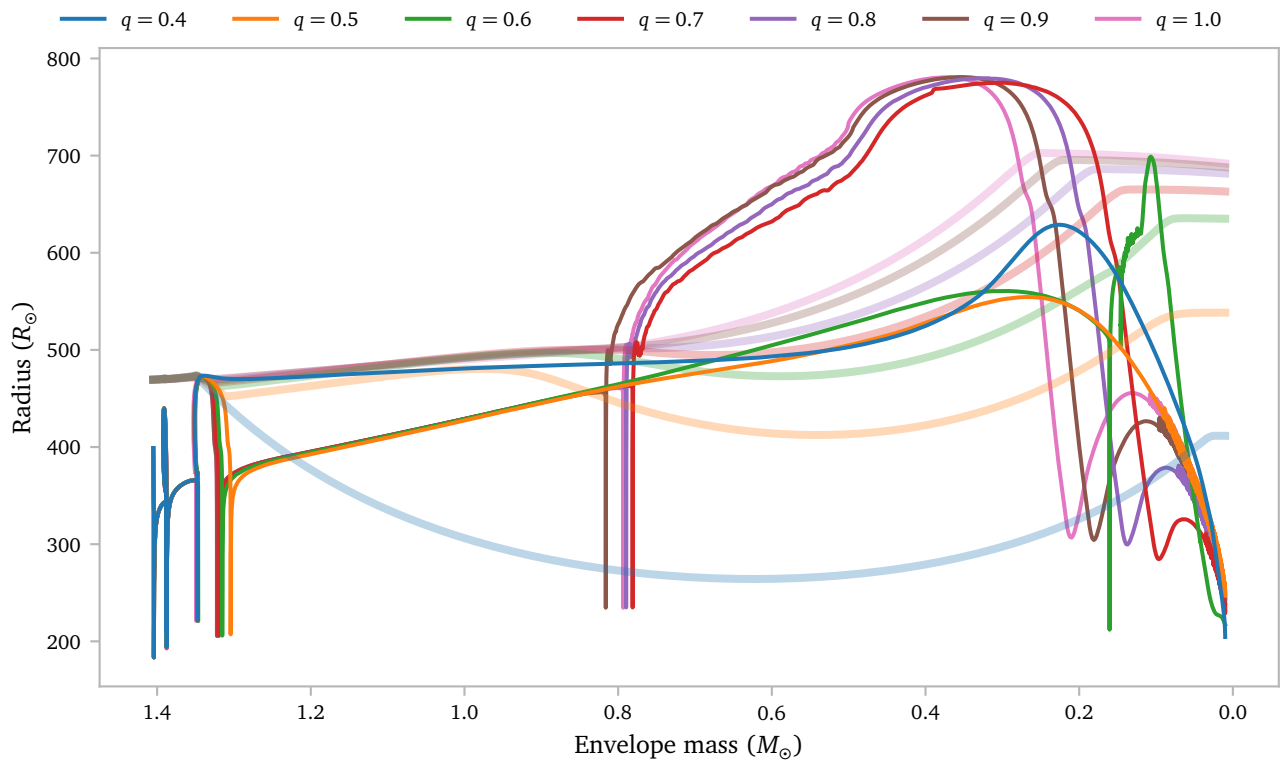
First I look into the two peculiar systems with $R_{\text{RL}} \approx 500$: It seems like the



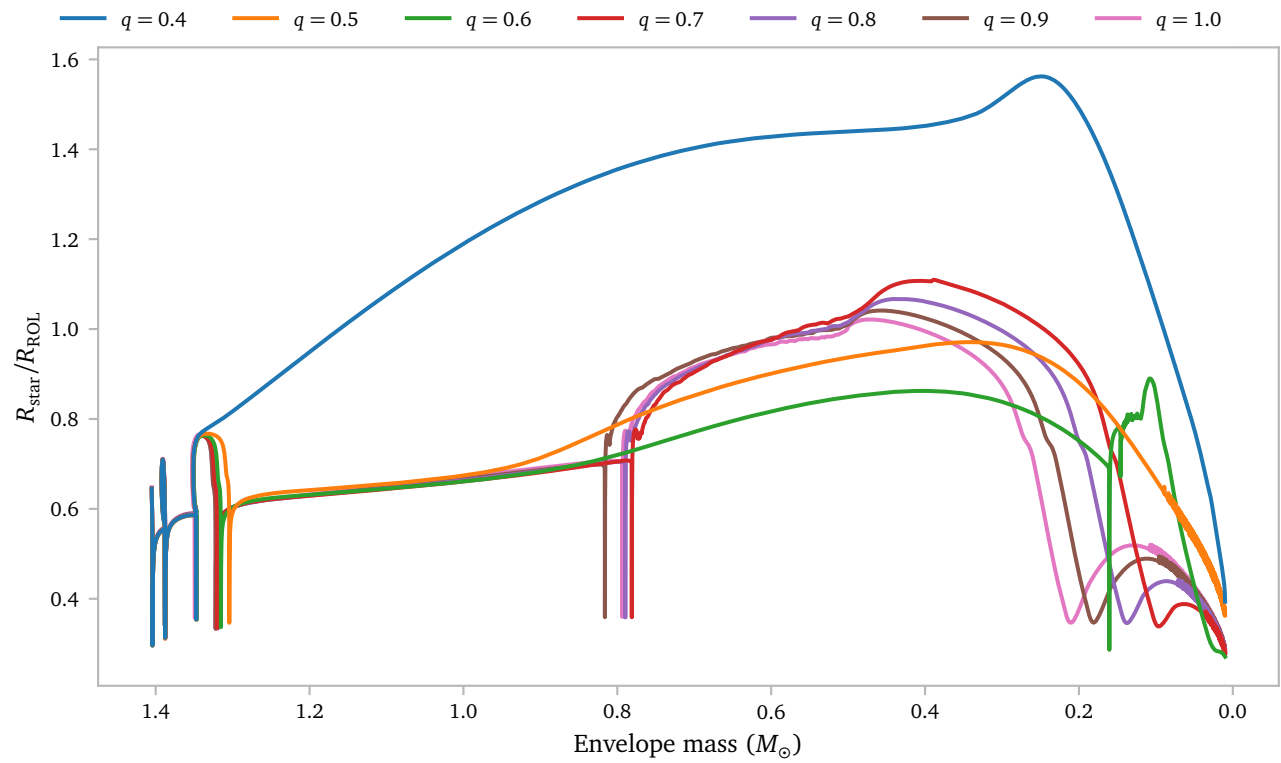
systems with $q = 0.5$ and 0.6 actually fill their Roche lobes just enough on during a previous pulse to alter their evolution afterwards. Most importantly, their next "pulse" occurs sooner and seems to be much less violent / abrupt.

The system with $q = 0.4$ actually loses all of its mass during the first pulse. That is why this system fills its outer lobe, just like the other ones.

The figure below shows this more clearly.

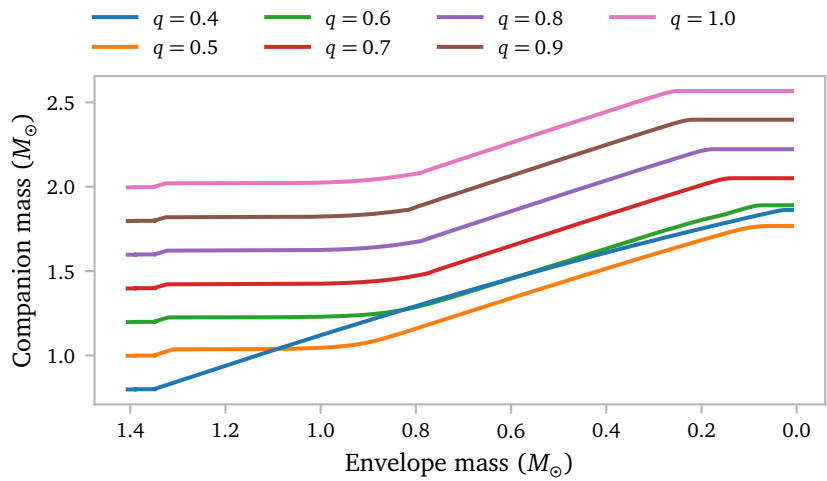


Below, I show the ratio between the star radius and the outer lobe. I am



really not too sure what is happening with the orange and green systems. It almost looks like they keep losing mass due to wind but I don't think that is actually the case.

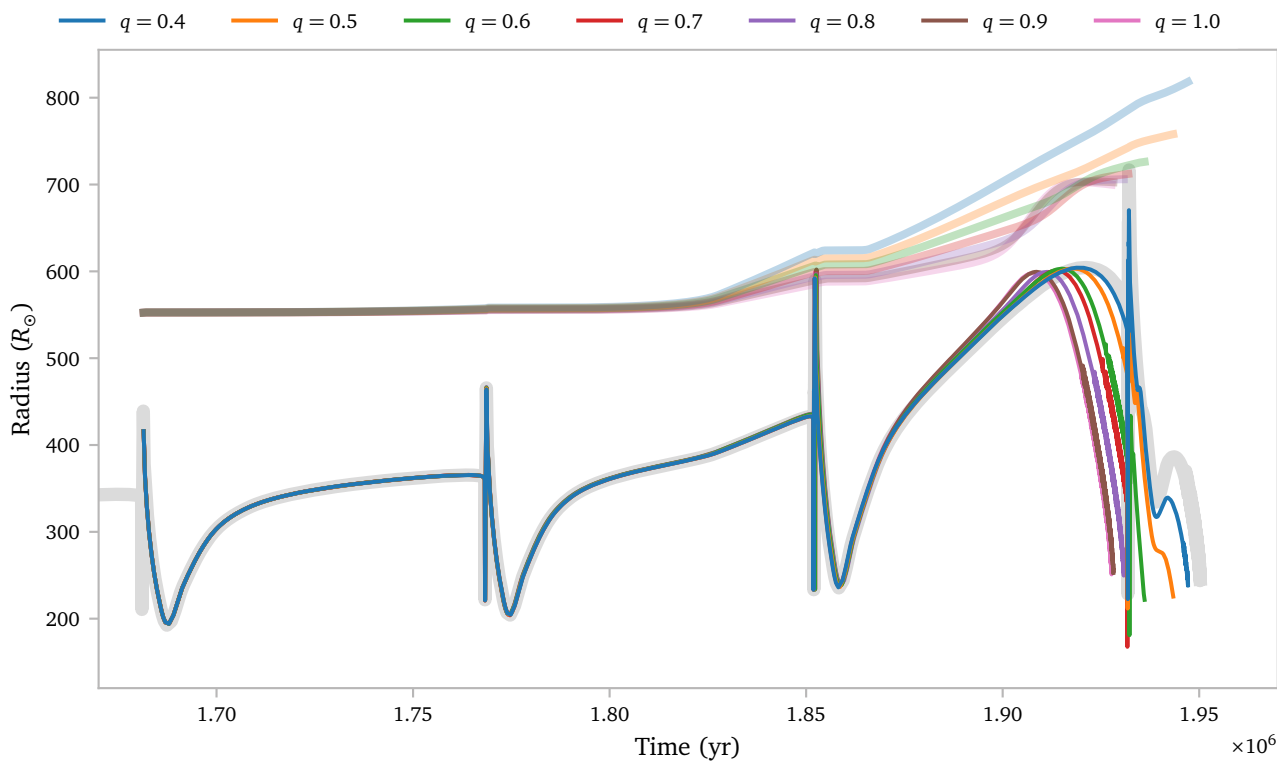
The plot below shows the mass of the companion star.



If the mass was actually lost due to wind, the companion would not gain significant amounts of mass.

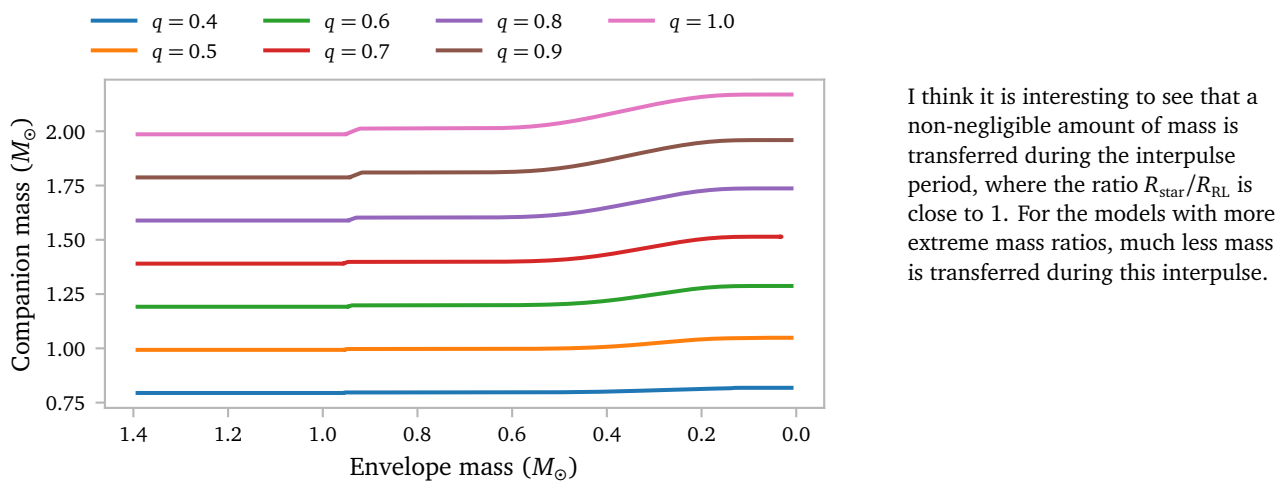
Now, I turn my attention to the last two columns of the grid figures. These models experience significant amounts of wind mass loss, and in the fast wind loss mode, this results in expanded orbits (and thus Roche lobes).

Below, the effects of this can clearly be seen for the models starting with $R_{\text{RL}} \approx 550R_{\odot}$.

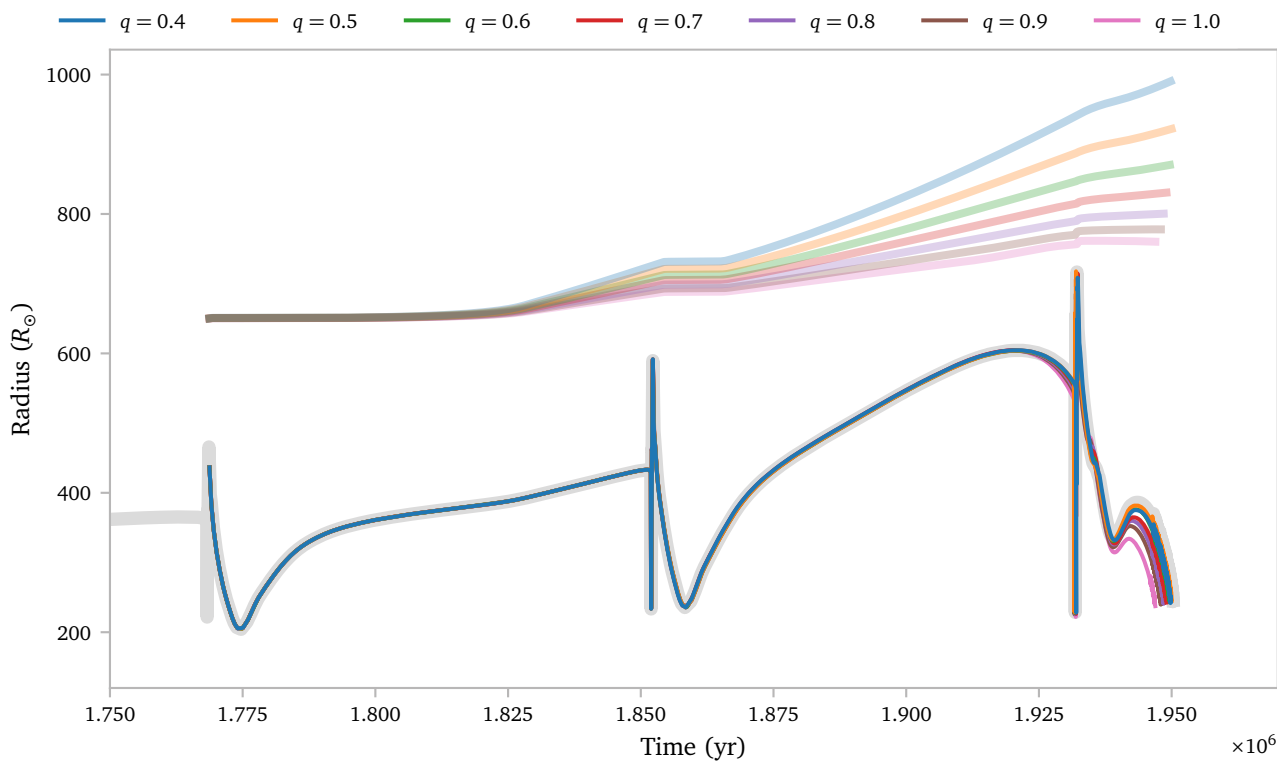


The models graze, or barely miss the second to last thermal pulse. They then continue and the evolution seems to be slightly altered for the models, depending on how much mass was transferred during the previous pulse.

Below, I show the mass of the companion star as a function of envelope mass again.



Then, lastly, I look at the evolution of the last column ($R_{\text{RL}} = 650R_{\odot}$).



Here, the wind manages to influence the orbits enough to avoid significant mass transfer in all cases.

c. Wind Loss

MESA accounts for orbital angular momentum changes using the `standard_jdot_ml` routine. Below, I show the full thing:

```
subroutine default_jdot_ml(binary_id, ierr)
  integer, intent(in) :: binary_id
  integer, intent(out) :: ierr
  type (binary_info), pointer :: b
  real(dp) :: alfa
  ierr = 0
  call binary_ptr(binary_id, b, ierr)
  if (ierr /= 0) then
    write(*,*) 'failed in binary_ptr'
    return
  end if
  !mass lost from vicinity of donor
  b% jdot_ml = (b% mdot_system_transfer(b% d_i) + b% mdot_system_wind(b% d_i))*&
    pow2(b% m(b% a_i)/(b% m(b% a_i)+b% m(b% d_i))*b% separation)*2*pi/b% period *&
    sqrt(1 - pow2(b% eccentricity))
  !mass lost from vicinity of accretor
  b% jdot_ml = b% jdot_ml + (b% mdot_system_transfer(b% a_i) + b% mdot_system_wind(b% a_i))*&
    pow2(b% m(b% d_i)/(b% m(b% a_i)+b% m(b% d_i))*b% separation)*2*pi/b% period *&
    sqrt(1 - pow2(b% eccentricity))
  !mass lost from circumbinary coplanar toroid
  b% jdot_ml = b% jdot_ml + b% mdot_system_cct * b% mass_transfer_gamma * &
    sqrt(standard_cgrav * (b% m(1) + b% m(2)) * b% separation)
end subroutine default_jdot_ml
```

The change in orbital angular momentum is clearly the sum of three terms. First, a term for mass loss from the donor. This term is quite similar to the fast winds approximation or the Jeans mode (Jeans, 1924), which is generally described using the specific angular momentum as:

$$\begin{aligned} h &= \frac{J}{M_d} = a_d^2 \omega = \left(\frac{M_a}{M_a + M_d} \right)^2 \sqrt{G(M_a + M_d)a} \\ &= \left(\frac{M_a a}{M_a + M_d} \right)^2 \sqrt{G \frac{M_a + M_d}{a^2}}. \end{aligned}$$

Now, applying Kepler’s third law, this reduces to

$$\begin{aligned} h &= \left(\frac{M_a a}{M_a + M_d} \right)^2 \frac{2\pi}{T} \\ \frac{dJ}{dt} &= h \frac{dM_d}{dt} = (\dot{M}_{d,wind} + \dot{M}_{d,transfer}) \left(\frac{M_a a}{M_a + M_d} \right)^2 \frac{2\pi}{T} \end{aligned}$$

In the absence of eccentricity, this is exactly the first equation in the MESA implementation. The second equation is simply the same, just now for the accretor.

The last equation is again just the Jeans mode, but now for material escaping from a circumbinary coplanar toroid². There are two new terms here though, $\gamma_{\text{mass transfer}}$ and $\dot{M}_{\text{cct}}$. The new term, $\gamma_{\text{mass transfer}}$ just determines the radius of the circumbinary coplanar toroid (cct), where $r_{\text{cct}} = \gamma_{\text{mass transfer}}^2 a_{\text{orb}}$. The term $\dot{M}_{\text{cct}}$ just determines the amount of mass that is lost from the cct and in mesa is defined as

² Read donut-shaped ring of gas lying in the same plane as the orbit.

```
b% mdot_system_cct = b% mtransfer_rate * b%
  _ mass_transfer_delta
```

and `b% mass_tranfer_delta` is the fraction of mass lost from circumbinary coplanar toroid.

For now, we assume fully conservative mass transfer, such that $\delta_{\text{mass transfer}} = 0$ so $\dot{M}_{\text{cct}} = 0$. Similarly, since we assume fully conservative mass transfer, $\dot{M}_{\text{transfer}} = 0$ for both the donor and the accretor. The accretor is modelled as a point mass and therefore does not lose any mass due to winds.

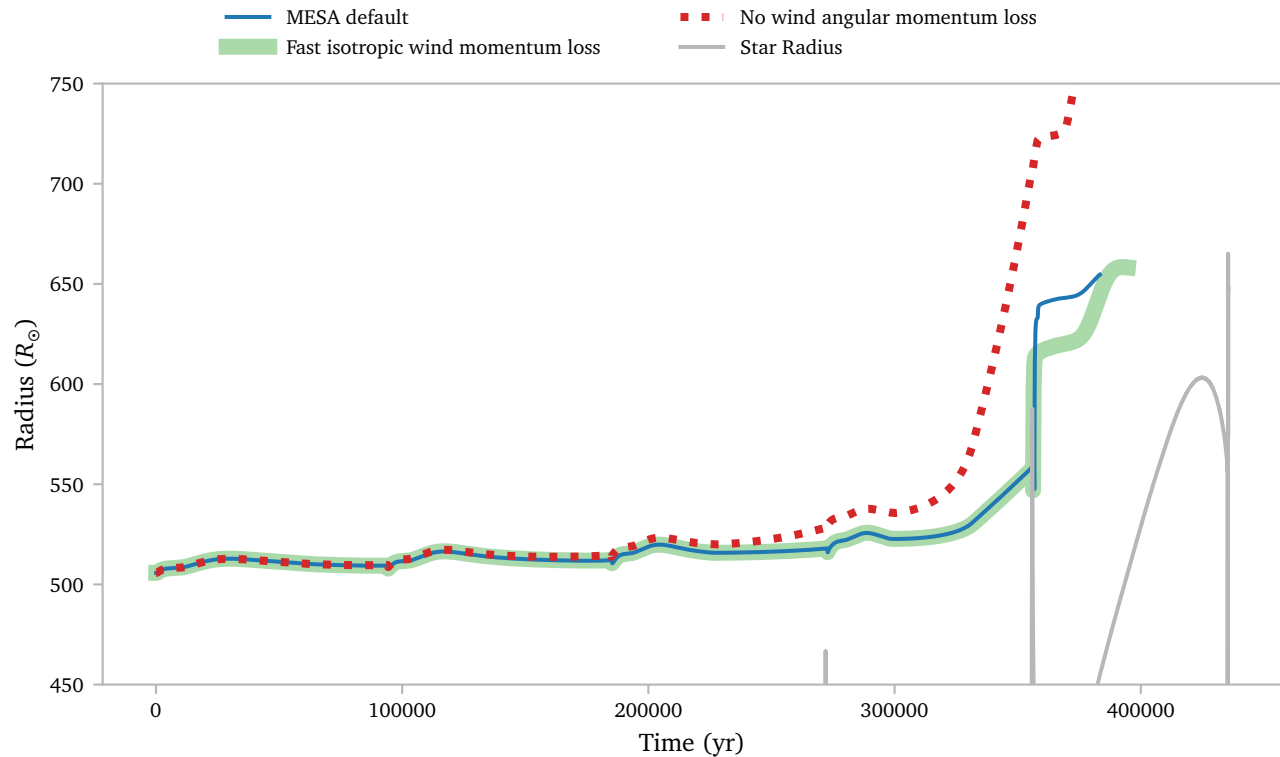
In our case, MESA’s implementation is really just

```
subroutine default_jdot_ml(binary_id, ierr)
  integer, intent(in) :: binary_id
  integer, intent(out) :: ierr
  type (binary_info), pointer :: b
  real(dp) :: alfa
  ierr = 0
  call binary_ptr(binary_id, b, ierr)
  if (ierr /= 0) then
    write(*,*) 'failed in binary_ptr'
    return
  end if
  !mass lost from vicinity of donor
  b% jdot_ml = b% mdot_system_wind(b% d_i) *&
    pow2(b% m(b% a_i)/(b% m(b% a_i)+b% m(b% d_i))*b% separation)*2*pi/b% period
end subroutine default_jdot_ml
```

The default MESA implementation of the angular momentum loss due to mass loss can be changed by using the `other_jdot_ml` hook and writing a custom routine to change `b% jdot_ml`. To start, I want to test a custom *no wind* and *simple fast wind* prescription.

Both are very simple. The first routine simply sets `b% jdot_ml = 0` in all cases. This can used as a test. In the case where the angular momentum of the orbit remains unchanged, one would expect the orbits to grow significantly more than in the case where the angular momentum of the orbit *shrinks*. So really, this test should give an upper bound of the orbital radius.

Similarly, I implement the simplified code I wrote above as a routine. This can also be used as a test. It *should* give the same results as the default MESA implementation.



We can quite clearly see that the *No angular momentum* model indeed leads to larger Roche lobe Radii (and orbital separations), while the *MESA default* is identical to the *Fast isotropic winds with momentum* loss up to the last pulse. This difference is not caused by wind implementation, but caused by the unstable implementation of the convective region determination that was still used for the default mesa model.

The `orbit_evol.py` file also implements the fast wind in a very similar way by calculating the change in orbital angular momentum as

```
# Change in semi-major axis, from angular momentum conservation.
dj_dt_orb = (
  (1 - beta)
  * eta
  * a**2
  * np.sqrt(1 - e**2)
  * omega_Kepler(SP.mass + mcomp, a)
  * SP.dM_dt
)
```

In the fast tides mode, $\beta = 0$ and $\eta = (1 + q)^{-2}$ with $q = M_d/M_s$. This thus reduces to

$$\frac{dJ}{dt} = \dot{M}_{d,wind} \left(\frac{M_d a}{M_s + M_d} \right)^2 \omega.$$

This is obviously the same result as the result from MESA. The cool thing however, is that this python file shows how we can modify β and η to use different wind angular momentum loss prescriptions. The most interesting one for us here is the prescription by Saladino et al. (2019).

The code from `orbit_evol.py` contains the following definitions for β and η .

```
def wind_loss(a, e, spin, mcomp, SP, EvPars):
  ...
  if EvPars.wind_model == "Saladino":
    # Accretion and AM loss from SPH simulations by Saladino et al (2019).
    # (N.B. model results and fit are only for circular orbits...)
    eta = eta_Sal(Q, vw_over_vorb)
    beta = beta_Sal(Q, vw_over_vorb)
  ...

def beta_Sal(q, v):
  """
  Fit to the accretion efficiency from the SPH simulation results of
  Saladino et al (2019) [eq.11 in the paper].
  q = M_loser/M_companion
  v = ratio of outflow velocity to orbital velocity
  """
  c1 = 1.7 + 0.3 * q
  c2 = 0.5 + 0.2 * q
  alpha = 1.0 / (c1 + (c2 * v) ** 5) + 0.75
  beta_max = min(0.3, 1.4 / q**2)
  beta = min(beta_BHL(q, v, alpha), beta_max)
  return beta

def beta_BHL(q, v, alpha=0.75):
  """
  Bondi-Hoyle-Lyttleton accretion efficiency, for a circular orbit.
  q = M_loser/M_companion
  v = ratio of outflow velocity to orbital velocity
  alpha = BHL efficiency parameter (0.75 by default)
  """
  vrel = np.sqrt(1 + v**2)
  beta = alpha / ((1 + q) ** 2 * v * vrel**3)
  return beta

def eta_Sal(q, v):
  """
  Specific orbital angular momentum loss (in units of J/mu = a^2 omega) fitted
  to the SPH simulation results of Saladino et al (2019) [eq.7 in the paper].
  q = M_loser/M_companion
  v = ratio of outflow velocity to orbital velocity
  """
  eta_iso = (1 + q) ** (-2)
  c1 = max(q, 0.6 * q**1.7)
  c2 = 1.5 + 0.3 * q
  eta = min(1.0 / (c1 + (c2 * v) ** 3) + eta_iso, 0.6)
  return eta
```

These functions are all pretty easy to implement in Fortran. For completeness, I wrote them at the end of this document. Below, I show the fortran subroutine that is responsible for the wind angular momentum change.

```
! src/bin/additional_routines.inc
subroutine wind_loss(binary_id, ierr)
  integer, intent(in) :: binary_id
  integer, intent(out) :: ierr
  type(binary_info), pointer :: b
  real(dp) :: q, vw_over_vorb
  real(dp) :: beta, eta
  ierr = 0
  call binary_ptr(binary_id, b, ierr)
  if (ierr /= 0) then
    write (*, *) 'failed in binary_ptr'
    return
  end if

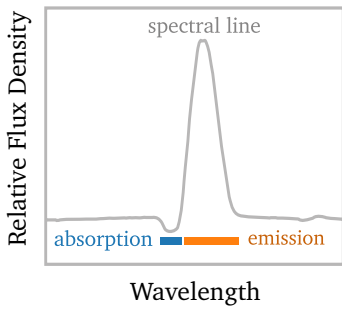
  ! --- compute q == m_accretor / m_donor and v
  q = b%m(b%a_i)/b%m(b%d_i)
  vw_over_vorb = compute_vwind(b%s_donor) / compute_vorbit(b% m(1) + b% m(2), b% separation)

  ! --- get beta and eta from Saladino
  beta = compute_beta_Sal(q, vw_over_vorb) / sqrt(1 - pow2(b%eccentricity))
  eta = compute_eta_Sal(q, vw_over_vorb)

  !mass lost from vicinity of donor
  b%jdot_ml = b%mdot_system_wind(b%d_i) &
    *(1 - beta) &
    *eta &
    *(b%separation)**2 &
    *sqrt(1 - pow2(b%eccentricity)) &
    *2*pi/b%period

end subroutine wind_loss
```

To complete the routine, we need a method to compute the wind velocity. The file `star_model.py` contains a method to compute the wind velocity based on results from Lamers et al. (1995). They determined the terminal velocities v_∞ of the stellar winds of 117 stars using 3 different methods, but most importantly by looking at the violet extend of the black absorption trough of saturated P Cygni profiles. These profiles are special as both an absorption dip, as well as an emission peak can be seen for the same spectral line. This is caused by Doppler shifting of the light, which can be used to derive the escape velocity. I show a schematic depiction in the margin (source: Wikimedia Commons (2024)).



Lamers et al. (1995) find a bistable relation between the escape velocity and the effective temperature of the star shown in figure 4. This relation can quite easily, but not extremely accurately be described by the following set of criteria:

$$v_\infty/v_{\text{esc}} = \begin{cases} 0.35 & \text{if } \log T_{\text{eff}} < 3.5, \\ 1.3 & \text{if } 4.0 \leq \log(T_{\text{eff}}) < 4.35, \\ 2.6 & \text{if } \log(T_{\text{eff}}) \geq 4.35, \\ 0.7 + 0.7(\log T_{\text{eff}} - 4.0) & \text{else.} \end{cases}$$

The Fortran implementation is written below.

An important aspect of winds that I have ignored up to now is the angular momentum loss associated with the mass loss. If we expand the equation for change in angular momentum, we can see the exact dependence:

$$\begin{aligned} \frac{dJ}{dt} &= \frac{d}{dt} (I\Omega) = \left(\frac{dJ}{dt} \right)_{\text{LS}} + \left(\frac{dJ}{dt} \right)_{\text{ML}} \\ I\dot{\Omega} + \Omega\dot{I} &= \dot{J}_{\text{LS}} + \dot{J}_{\text{ML}} \\ \dot{\Omega} &= \frac{\dot{J}_{\text{LS}}}{I} + \frac{\dot{J}_{\text{ML}}}{I} - \frac{\dot{I}}{I} \dot{I} \\ \dot{\Omega} &= \underbrace{\frac{\dot{\Omega}_{\text{LS}}}{I}}_{\text{tides}} + \underbrace{\frac{\dot{J}_{\text{ML}}}{I}}_{\text{mass loss}} - \underbrace{\frac{\dot{I}}{I}}_{\text{structure}} \end{aligned}$$

The tides term $\dot{\Omega}_{\text{LS}}$ is described by the Hut (1981) formalism:

$$\dot{\Omega}_{\text{LS}} = 3 \frac{k}{T} \frac{q^2}{r_g^2} \left(\frac{R}{a} \right)^6 (\Omega_{\text{orb}} - \Omega).$$

The mass loss term can be derived from isotropic mass loss. The specific angular momentum of a mass element at the surface of a star rotating at an angular velocity Ω is given by:

$$\begin{aligned} \vec{h} &= \vec{r} \times \vec{v} = R \hat{r} \times (\vec{\Omega} \times \vec{r}) \\ &= R \hat{r} \times (\Omega \hat{z} \times R \hat{r}) \\ &= \Omega R^2 \hat{r} \times (\hat{z} \times \hat{r}) \\ &= \Omega R^2 [\hat{z} (\hat{r} \cdot \hat{r}) - \hat{r} (\hat{r} \cdot \hat{z})] \\ &= \Omega R^2 [\hat{z} - \cos \theta \hat{r}]. \end{aligned}$$

We are only interested in the specific angular momentum about the axis of rotation $\hat{z}$:

$$\begin{aligned} h_z &= \vec{h} \cdot \hat{z} = \Omega R^2 [1 - \cos^2 \theta] \\ &= \Omega R^2 \sin^2 \theta \\ \langle h_z \rangle &= \Omega R^2 \langle \sin^2 \theta \rangle = 2\Omega R^2/3. \end{aligned}$$

The change in angular momentum is than quite easily computed by multiplying by the wind mass loss rate:

$$\frac{\dot{J}_{\text{ML}}}{I} = \frac{2\Omega R^2 \dot{M}_{\text{wind}}}{3I}.$$

Lastly, the structure term makes sure the rotation of the star responds to structural changes such as radial expansion or mass loss:

$$\Omega \dot{I} = \Omega \frac{d \ln I}{dt}.$$

All of the rotation physics is implemented in the tides subroutine³. Below, I show the part of the routine responsible for the stellar rotation.

³ I am not happy with this, but I am also not sure where and how I should update the stellar rotation consistently other than in the tides subroutine.

```
! src/bin/additional_routines.inc
subroutine tides(binary_id, ierr)
  ...
  ! --- get initial spin and orbital frequency
  omega = s_d%x_ctrl(3)
  omega_orb = 2d0*pi/b%period

  ! --- compute moment of inertia of solid body and gyration radius
  I_eff = compute_I_eff(s_d)
  rg2 = compute_rg2(s_d, I_eff)

  ! --- angular momentum loss due to structural change
  I_old = s_d%x_ctrl(4)
  if (I_old > 0d0 .and. dt > 0d0) then
    dlnI_dt = (log(I_eff) - log(I_old))/dt
  else
    dlnI_dt = 0d0
  end if
  domega_dt_structure = -omega*dlnI_dt

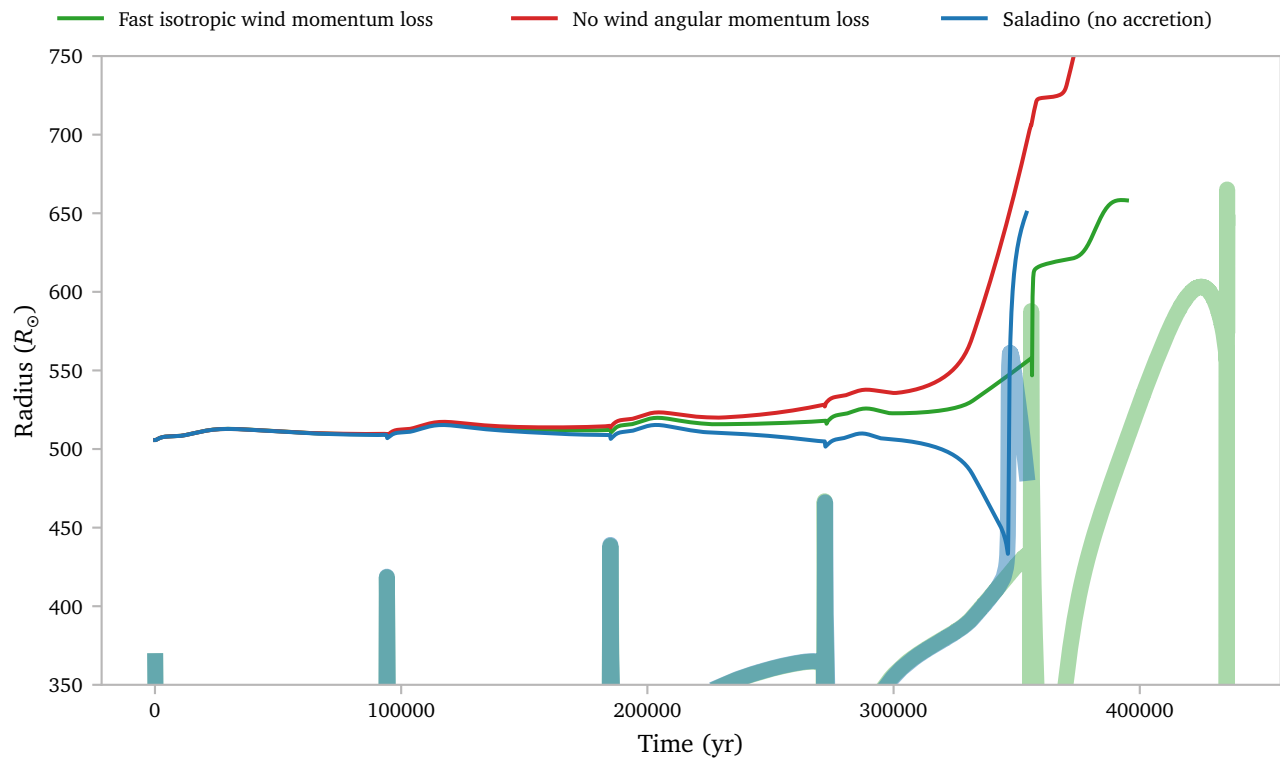
  ! --- hut spin evolution
  domega_dt_tides = 3*k_over_T*q**2/(rg2)*(R/a)**6*(omega_orb - omega)

  ! --- angular momentum loss due to winds
  dj_dt_rot = b%mdot_system_wind(b%d_i)*R**2*omega/1.5
  domega_dt_wind = dj_dt_rot/I_eff

  ! --- total spin evolution (structure + wind + tides)
  domega_dt = domega_dt_structure + domega_dt_wind + domega_dt_tides

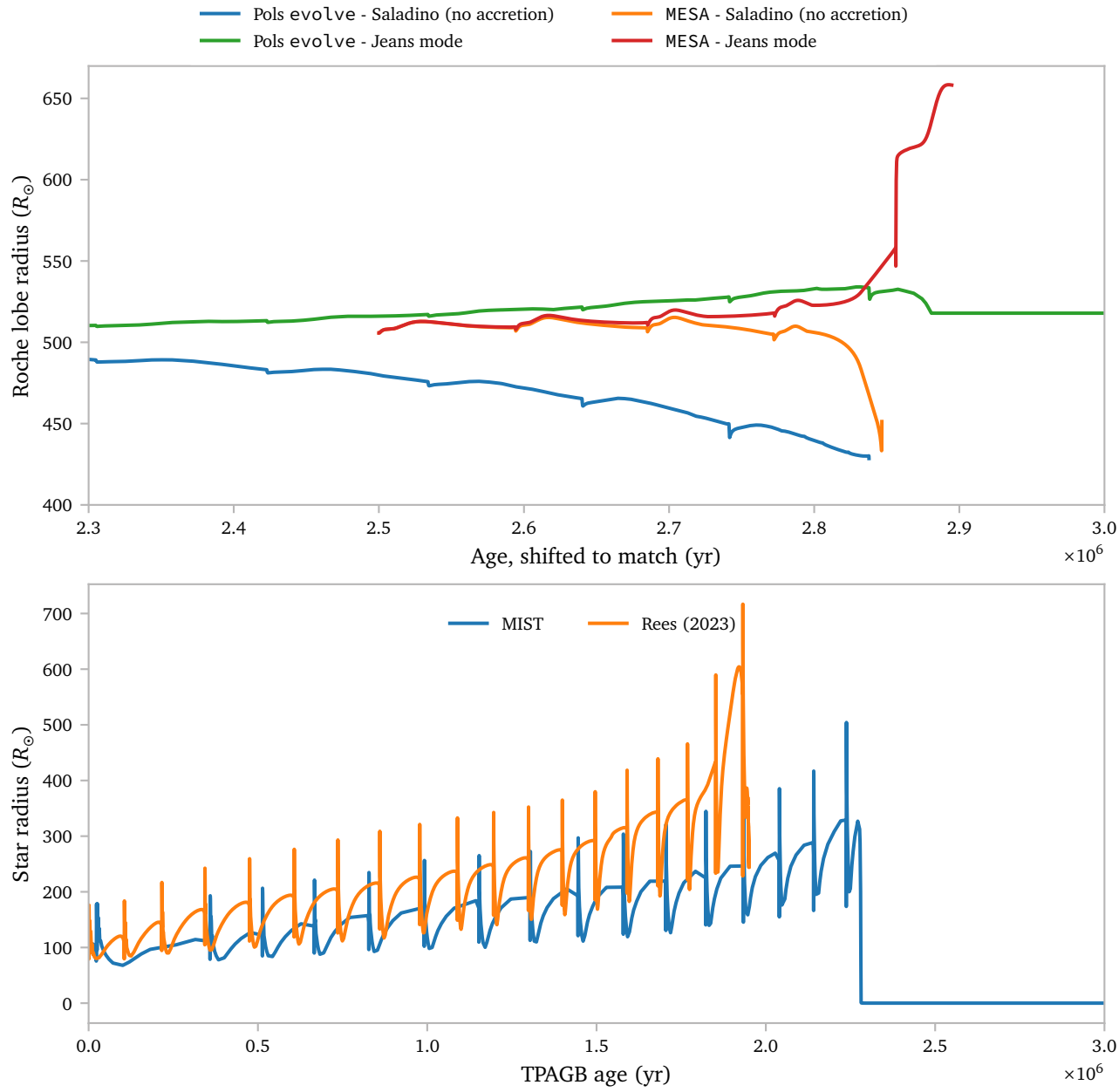
  ! --- update spin
  omega = omega + dt*domega_dt
  s_d%x_ctrl(3) = omega
  ...
end subroutine tides
```

Below, I plot the results from a star using this prescription, Saladino (no accretion).



Interestingly, the Saladino prescription actually forces the star to fill its Roche lobe during the thermal pulse. Without the wind angular momentum loss, a $2M_\odot$ star will *only* fill its Roche lobe during a thermal pulse.

I also compare these results with the MIST models used in the `evolve.py`. For this comparison, I chose $q = 1.25, a = 1400M_{\odot}, e = 0^4$.



I think⁵ the differences in the changes in orbit between the python code and the MESA routines can be explained by the differences in the single star model. Since the MIST models used in the python code tend to have significantly smaller radii, the bumps in the orbital separation (and thus Roche lobe) will be less pronounced in the MIST models. Since the MIST models use the Böcker wind, which ramps up much more smoothly than the Vassiliadis & Wood (1993) winds used for the MESA models, the mass loss is also less extreme in and more gradual in the MIST models then in the MESA models. In the MESA models, the onset of significant wind mass loss can actually quite clearly be seen in both the stellar radius evolution, as well as in the Roche lobe evolution.

Right now, I am assuming wind is lost from the system, while I am using the angular momentum prescription from Saladino et al. (2019). This prescription assumes a fraction β of the wind mass is accreted onto the accretor. Obviously, to accurately model the tidal evolution, it is quite important to take this effect into account.

MESA has a built in *wind accretion* prescription based on the Bondi-Hoyle approximation as described in Hurley et al. (2002).

```
! $MESA_DIR/binary/private/binary_wind.f90
subroutine Bondi_Hoyle_wind_transfer(binary_id, s_i, ierr)
  integer, intent(in) :: binary_id, s_i ! s_i is index of the wind mass losing star
  integer, intent(out) :: ierr

  ! wind transfer fraction based on Bondi-Hoyle mechanism as described in
  ! Hurley et al. 2002, MNRAS, 329, 897-928

  type(binary_info), pointer :: b
  type (star_info), pointer :: s
  real(dp) :: v_orb, v_wind
  real(dp) :: alpha, beta ! Bondi-Hoyle alpha, beta for that star
  real(dp) :: max_xfer ! Maximum transfer fraction

  call binary_ptr(binary_id, b, ierr)
  if (ierr /= 0) then
    write(*,*) 'failed in binary_ptr'
    return
  end if

  if (s_i == 1) then
    s => b% s1
    alpha = b% wind_BH_alpha_1
    beta = b% wind_BH_beta_1
    max_xfer = b% max_wind_transfer_fraction_1
  else
    s => b% s2
    alpha = b% wind_BH_alpha_2
    beta = b% wind_BH_beta_2
    max_xfer = b% max_wind_transfer_fraction_2
  end if

  ! orbital speed Hurley et al 2002 eq. 8
  v_orb = sqrt(standard_cgrav * (b% m(1) + b% m(2)) / b% separation) !cm/s

  ! windspeed from Hurley et al 2002 eq. 9
  v_wind = sqrt(2d0 * beta * standard_cgrav * b% m(s_i) / b% r(s_i))

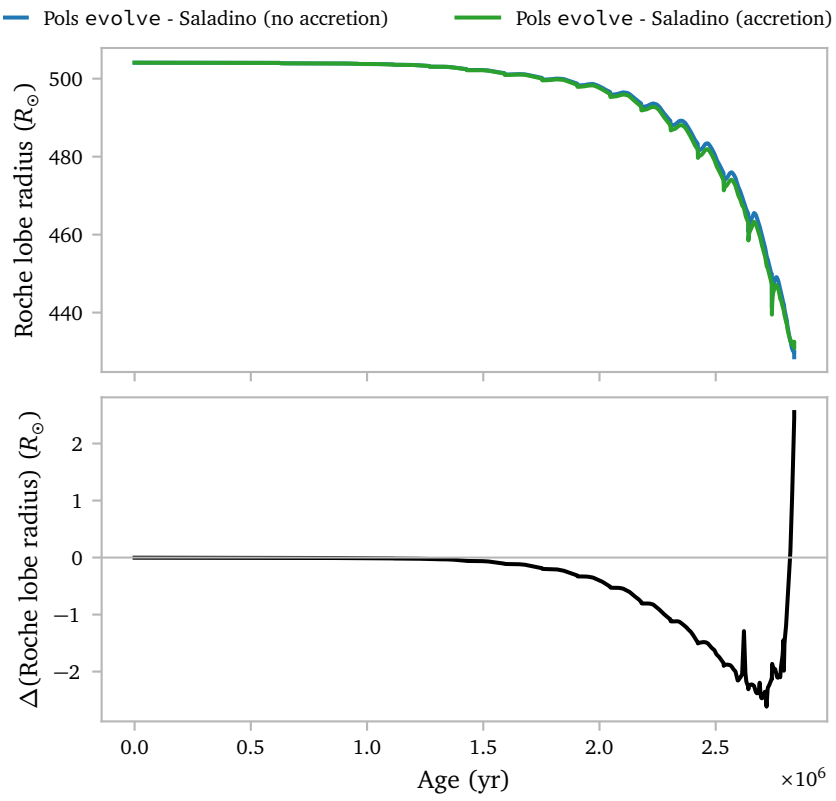
  ! Bondi-Hoyle transfer fraction Hurley et al. 2002 eq. 6
  b% wind_xfer_fraction(s_i) = alpha / pow2(b% separation) /&
    (2d0 * sqrt(1d0 - pow2(b% eccentricity))) *&
    pow2(standard_cgrav * b% m(3-s_i) / pow2(v_wind)) *&
    pow(1d0 + pow2(v_orb/v_wind),-1.5d0)

  ! limit to provided maximum
  b% wind_xfer_fraction(s_i) = min(max_xfer, b% wind_xfer_fraction(s_i))

end subroutine Bondi_Hoyle_wind_transfer
```

This built in prescription can however quite easily be swapped with a user defined model using the `use_other_binary_wind_transfer` option. I think this prescription should be pretty straightforward using the functions I have already defined for the wind angular momentum loss⁶.

To test the impact of this change, we can use the python code and simply choose to account for the change in mass of the companion due to winds or ignore it, just like in my MESA code. The differences are shown below.



⁴ This code assumes $q = M_d/M_s$, while I use the other definition.

⁵ But I am not sure, maybe I could use my single star TPAGB models as input in the python code.

⁶ But I have not had time to properly implement and test this yet.

The differences are quite small, but not negligible. It is still important to take this effect into account and implement a solid wind mass transfer mechanism based on the results from Saladino et al. (2019).

d. Analytical Period, Mass-Ratio, Roche-Lobe Radius Relations

I have not looked at this yet, as the debugging of the tides prescription and the implementation of the wind mass transfer took more time than expected.

References

Hurley, J. R., Tout, C. A., & Pols, O. R. (2002) *Evolution of binary stars and the effect of tides on binary populations*. [MNRAS329](#), 897.

Hut, P. (1981) *Tidal evolution in close binary systems..* [A&A99](#), 126.

Jeans, J. H. (1924) *Cosmogonic problems associated with a secular decrease of mass*. [MNRAS85](#), 2.

Lamers, H. J. G. L. M., Snow, T. P., & Lindholm, D. M. (1995) *Terminal Velocities and the Bistability of Stellar Winds*. [ApJ455](#), 269.

Saladino, M. I., Pols, O. R., & Abate, C. (2019) *Slowly, slowly in the wind. 3D hydrodynamical simulations of wind mass transfer and angular-momentum loss in AGB binary systems*. [A&A626](#), A68.

Vassiliadis, E. & Wood, P. R. (1993) *Evolution of Low- and Intermediate-Mass Stars to the End of the Asymptotic Giant Branch with Mass Loss*. [ApJ413](#), 641.

Wikimedia Commons. (2024) *File:P Cygni Profile-en.svg* — *Wikimedia Commons, the free media repository*. [Online; accessed 29-March-2026].

e. Fortran Functions

```
! src/bin/additional_routines.inc
real(dp) function compute_vwind(s) result(vwind)
    implicit none

    type(star_info), pointer :: s
    real(dp) :: m, R, log_Teff
    real(dp) :: v_esc, vw_factor

    m = s% star_mass * Msun
    R = s%photosphere_r * Rsun
    log_Teff = log10(s% Teff)

    v_esc = sqrt(2.0 * standard_cgrav * m / R)
    if (log_Teff < 3.5d0) then
        vw_factor = 0.35d0
    else if (log_Teff <= 4.0d0) then
        vw_factor = 0.7d0 + 0.7d0 * (log_Teff - 4.0d0)
    else if (log_Teff <= 4.35d0) then
        vw_factor = 1.3d0
    else
        vw_factor = 2.6d0
    end if
    vwind = vw_factor * v_esc
end function compute_vwind

real(dp) function compute_vorbit(Mtot, a) result(vorbit)
    implicit none

    real(dp) :: Mtot, a
    vorbit = sqrt( standard_cgrav * Mtot / a)

end function compute_vorbit

real(dp) function compute_beta_Sal(q, v) result(beta)
    implicit none

    real(dp) :: q, v
    real(dp) :: c1, c2
    real(dp) :: alpha, beta_max

    c1 = 1.7 + 0.3/q
    c2 = 0.5 + 0.2/q
    alpha = 1.0/(c1 + (c2*v)**5.0) + 0.75
    beta_max = min(0.3d0, 1.4*q**2)
    beta = min(compute_beta_BHL(q, v, alpha), beta_max)

end function compute_beta_Sal

real(dp) function compute_beta_BHL(q, v, alpha) result(beta)
    implicit none

    real(dp) :: q, v, alpha
    real(dp) :: vrel

    vrel = sqrt(1 + v**2)
    beta = alpha/((1 + 1/q)**2*v * vrel**3)

end function compute_beta_BHL

real(dp) function compute_eta_Sal(q, v) result(eta)
    implicit none

    real(dp) :: q, v
    real(dp) :: eta_iso, c1, c2

    eta_iso = (1 + 1/q)**(-2)
    c1 = max(1/q, 0.6*(1/q)**1.7)
    c2 = 1.5 + 0.3/q
    eta = min(1.0/(c1 + (c2*v)**3) + eta_iso, 0.6d0)

end function compute_eta_Sal
```